

FootLel — Código Base Backend

Estructura de Arquitectura en Capas en Python + FastAPI según Lineamientos de la Cátedra

Cumplimiento de Pautas de Cátedra (Clase 7)

- **routes/controllers:** Reciben la petición HTTP y derivan al servicio correspondiente (sin lógica de negocio).
- **services:** Contiene la lógica de negocio y validaciones del dominio de FootLel.
- **models/repositories:** Conexión centralizada a base de datos y consultas SQL.
- **Manejo estandarizado de excepciones:** Respuestas de error compuestas por code, message y detail.
- **Documentación automática:** Swagger UI integrado automáticamente con FastAPI en /docs.

1. Estructura del Proyecto

```
FootLel-Backend/ ├── config/ | ├── __init__.py | └── database.py # Conexión centralizada a
SQLite / DB ├── controllers/ | ├── __init__.py | └── game_controller.py # Endpoints del
Juego / Jugadores ├── models/ | ├── __init__.py | └── game_models.py # Modelos SQLAlchemy
para la BD ├── repositories/ | ├── __init__.py | └── game_repository.py # Consultas directas
a la BD ├── services/ | ├── __init__.py | └── game_service.py # Reglas y Lógica de Negocio
└── schemas/ | ├── __init__.py | └── game_schemas.py # Schemas Pydantic para Swagger
└── .env.example # Plantilla de variables de entorno ├── .gitignore # Exclusión de BD, envs y
cachés └── main.py # Punto de entrada de FastAPI └── requirements.txt # Dependencias del
proyecto
```

2. Código Fuente por Módulo

requirements.txt

Dependencias

```
fastapi==0.110.0
uvicorn==0.28.0
sqlalchemy==2.0.28
pydantic==2.6.4
python-dotenv==1.0.1
```

.env.example

Variables de Entorno

```
# Configuración del servidor y Base de Datos
DATABASE_URL=sqlite:///./footlel.db
SECRET_KEY=clave_super_secreta_footlel_2026
DEBUG_MODE=True
```

config/database.py

Conexión BD

```

import os
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
from dotenv import load_dotenv

load_dotenv()

DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./footle1.db")

# Conexión única a la Base de Datos según buenas prácticas de la cátedra
engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False} if "sqlite" in DATABASE_URL else {}
)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    # Provee una sesión de base de datos por request y la cierra al finalizar
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

models/game_models.py

SQLAlchemy Models

```

from sqlalchemy import Column, Integer, String, DateTime
from datetime import datetime
from config.database import Base

class UsuarioModel(Base):
    __tablename__ = "usuarios"

    id = Column(Integer, primary_key=True, index=True)
    nombre_usuario = Column(String, unique=True, index=True, nullable=False)
    email = Column(String, unique=True, index=True, nullable=False)
    password_hash = Column(String, nullable=False)
    puntos = Column(Integer, default=0)

class PartidaModel(Base):
    __tablename__ = "partidas"

    id = Column(Integer, primary_key=True, index=True)
    jugador1_id = Column(Integer, nullable=False)
    jugador2_id = Column(Integer, nullable=True)
    estado = Column(String, default="PENDIENTE") # PENDIENTE, EN_CURSO, FINALIZADA
    ganador_id = Column(Integer, nullable=True)
    fecha_creacion = Column(DateTime, default=datetime.utcnow)

```

schemas/game_schemas.py

Pydantic / Swagger

```

from pydantic import BaseModel, EmailStr
from typing import Optional
from datetime import datetime

class UsuarioCreateRequest(BaseModel):
    nombre_usuario: str
    email: EmailStr
    password: str

class UsuarioResponse(BaseModel):
    id: int
    nombre_usuario: str
    email: str
    puntos: int

    class Config:
        from_attributes = True

class PartidaCreateRequest(BaseModel):
    jugador1_id: int

class PartidaResponse(BaseModel):
    id: int
    jugador1_id: int
    jugador2_id: Optional[int] = None
    estado: str
    ganador_id: Optional[int] = None
    fecha_creacion: datetime

    class Config:
        from_attributes = True

class ErrorStandardResponse(BaseModel):
    code: int
    message: str
    detail: str

```

repositories/game_repository.py

[Acceso a Datos](#)

```

from sqlalchemy.orm import Session
from models.game_models import UsuarioModel, PartidaModel

class GameRepository:
    def __init__(self, db: Session):
        self._db = db # Encapsulamiento de la sesión de BD

    def obtener_usuario_por_email(self, email: str):
        return self._db.query(UsuarioModel).filter(UsuarioModel.email == email).first()

    def obtener_usuario_por_username(self, username: str):
        return self._db.query(UsuarioModel).filter(UsuarioModel.nombre_usuario ==
username).first()

    def crear_usuario(self, usuario: UsuarioModel) -> UsuarioModel:

```

```

        self._db.add(usuario)
        self._db.commit()
        self._db.refresh(usuario)
        return usuario

    def crear_partida(self, partida: PartidaModel) -> PartidaModel:
        self._db.add(partida)
        self._db.commit()
        self._db.refresh(partida)
        return partida

    def obtener_partida_por_id(self, partida_id: int):
        return self._db.query(PartidaModel).filter(PartidaModel.id == partida_id).first()

```

services/game_service.py

Lógica de Negocio

```

from sqlalchemy.orm import Session
from repositories.game_repository import GameRepository
from models.game_models import UsuarioModel, PartidaModel
from schemas.game_schemas import UsuarioCreateRequest, PartidaCreateRequest
from fastapi import HTTPException, status

class GameService:
    def __init__(self, db: Session):
        self.repo = GameRepository(db)

    def registrar_usuario(self, datos: UsuarioCreateRequest) -> UsuarioModel:
        # Validaciones de regla de negocio
        if self.repo.obtener_usuario_por_email(datos.email):
            raise HTTPException(
                status_code=status.HTTP_409_CONFLICT,
                detail="El email ya se encuentra registrado en FootLe1."
            )

        if self.repo.obtener_usuario_por_username(datos.nombre_usuario):
            raise HTTPException(
                status_code=status.HTTP_409_CONFLICT,
                detail="El nombre de usuario ya está en uso."
            )

        nuevo_usuario = UsuarioModel(
            nombre_usuario=datos.nombre_usuario,
            email=datos.email,
            password_hash=f"hash_{datos.password}" # Reemplazar con hash real (ej. bcrypt)
        )
        return self.repo.crear_usuario(nuevo_usuario)

    def crear_nueva_partida(self, datos: PartidaCreateRequest) -> PartidaModel:
        nueva_partida = PartidaModel(jugador1_id=datos.jugador1_id)
        return self.repo.crear_partida(nueva_partida)

```

controllers/game_controller.py

Controladores / Rutas

```

from fastapi import APIRouter, Depends, status
from sqlalchemy.orm import Session
from config.database import get_db
from services.game_service import GameService
from schemas.game_schemas import (
    UsuarioCreateRequest, UsuarioResponse,
    PartidaCreateRequest, PartidaResponse,
    ErrorStandardResponse
)

router = APIRouter(prefix="/api", tags=["FootLel API"])

@router.post(
    "/usuarios",
    response_model=UsuarioResponse,
    status_code=status.HTTP_201_CREATED,
    responses={409: {"model": ErrorStandardResponse}}
)
def registrar_usuario(datos: UsuarioCreateRequest, db: Session = Depends(get_db)):
    # Registra un nuevo usuario en el juego FootLel. Valida duplicados.
    service = GameService(db)
    return service.registrar_usuario(datos)

@router.post(
    "/partidas",
    response_model=PartidaResponse,
    status_code=status.HTTP_201_CREATED
)
def crear_partida(datos: PartidaCreateRequest, db: Session = Depends(get_db)):
    # Crea una nueva sala o partida de juego
    service = GameService(db)
    return service.crear_nueva_partida(datos)

```

main.py

Entry Point

```

from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
from fastapi.middleware.cors import CORSMiddleware
from config.database import Base, engine
from controllers.game_controller import router as game_router

# Inicialización de la base de datos (Crea las tablas automáticamente)
Base.metadata.create_all(bind=engine)

app = FastAPI(
    title="FootLel API",
    version="1.0.0",
    description="Backend oficial del proyecto FootLel – Universidad de Belgrano",
    docs_url="/docs",          # Swagger UI
    redoc_url="/redoc"         # ReDoc alternativa
)

# Configuración de CORS para permitir peticiones desde el FrontEnd
app.add_middleware(

```

```

    CORSMiddleware,
    allow_origins=["*"], # En producción colocar la URL específica del FrontEnd
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Manejo global de excepciones para mantener formato uniforme de respuesta
@app.exception_handler(Exception)
async def global_exception_handler(request: Request, exc: Exception):
    return JSONResponse(
        status_code=500,
        content={
            "code": 500,
            "message": "Error interno del servidor",
            "detail": str(exc)
        }
    )

# Inclusión de rutas
app.include_router(game_router)

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)

```